Data Structures (ETCCDS202)

Unit–4

Assignment: Non-Linear & Advanced Data Structures

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech (CSE-AI/ML)

Semester: 1 Batch: 2025-29

Course Title: Data Structures

Course Code: ETCCDS202

Student Name: Payal

Roll No.: 2501730224

Section: A

Report

Task 1: Binary Search Tree (BST)

Question:

Implement a Binary Search Tree with insert, search, delete, and inorder traversal operations. Also test different deletion cases.

Answer:

A Binary Search Tree (BST) is a hierarchical data structure where each node has at most two children. The left subtree contains values smaller than the root, and the right subtree contains larger values.

Operations Implemented:

- Insert elements
- Search elements
- Delete nodes (leaf, one child, two children)
- Inorder traversal

Explanation:

Inorder traversal of a BST always prints elements in sorted order because it follows the Left → Root → Right sequence.

Time Complexity:

- Average: O(log n)
- Worst Case: O(n) (when tree becomes skewed)

Task 2: Graph (BFS & DFS)

Question:

Represent a graph using adjacency list and perform BFS and DFS traversal.

Answer:

A graph is a collection of nodes connected by edges. It is used in real-world systems like social networks, maps, and routing.

BFS (Breadth First Search):

- Uses queue
- Traverses level by level

DFS (Depth First Search):

- Uses recursion/stack
- Goes deep before backtracking

Difference:

- BFS finds shortest path (unweighted graph)
- DFS is useful for cycle detection

Real-world Use:

- BFS → shortest path (Google Maps)
- DFS → detecting loops in networks

Task 3: Hash Table

Question:

Implement hash table using separate chaining and demonstrate collision handling.

Answer:

A hash table stores key-value pairs and provides fast access using a hash function.

Collision:

When two keys map to same index.

Solution:

Separate chaining (store multiple values in same bucket using list)

Time Complexity:

- Average: O(1)
- Worst: O(n)